

ArtsyPhartsy Bag Uploader

Technical Reference & Design Guide

Version 2

May 2026

What this tool does

Watches three input folders for product photos, uses AI to detect and crop individual items, transforms them with professional studio photography (puffing bags, rendering aprons on mannequins, replacing backgrounds), then automatically creates draft WooCommerce products complete with names, descriptions, brand stories, pricing, SKUs, and dual images — finishing with an email notification.

Open source — MIT License — contributions welcome

What's New in Version 2

This version includes significant improvements over v1, primarily driven by the switch to gpt-image-2 and new image processing features.

Change	Details
gpt-image-2 for all image ops	Replaced gpt-image-1 throughout. Reasoning capability means tag text is now preserved correctly — the model actually follows instructions rather than generating plausible-looking but wrong text.
Background replacement	New prepareSecondaryImage() sends the flat crop to gpt-image-2 to replace the backdrop with a clean neutral gray studio background, matching the puffed hero image.
Presentation variety	puffBag() randomly selects from 6 handle/angle styles so bags don't all look identical on the store.
Brand story per bag	generateBrandStory() uses GPT-4o to write a slightly varied version of the brand story for each bag, with field validation to catch unexpected AI responses.
WooCommerce layout fix	Brand story moved to short_description (right panel above Add to Cart) matching the existing store layout.
Upload retry logic	uploadImage() retries up to 3 times on ETIMEDOUT or server errors with progressive 3s/6s/9s delays.
Primary fail protection	If primary image upload fails after all retries, secondary upload and product creation are both skipped — no orphaned products or mismatched images.
Karis link	Purpose field in brand story includes a clickable link to karis-society.org/about/
MIME type fix	tmp PNG path now uses <code>\/.[^.]++\$/</code> regex to handle any extension, fixing unsupported mimetype errors.

1. Project Overview

The ArtsyPhartsy Bag Uploader is a Node.js automation tool running on WSL (Windows Subsystem for Linux). Drop a photo of one or more bags into an input folder and the tool handles detection, AI image enhancement, and WooCommerce draft listing creation automatically.

1.1 Project Structure

File	Purpose
src/index.js	Folder watcher, queue manager, graceful shutdown on Ctrl+C
src/pipeline.js	Orchestrates the 4-step pipeline per photo
src/ai.js	All OpenAI API calls: detect, crop, puff, brand story, background replacement
src/woocommerce.js	Name uniqueness, image upload with retry, product creation
src/email.js	Notification email when drafts are ready
src/logger.js	Winston logging to console and file
.env	All credentials and configuration — never commit to git
logs/uploader.log	Full run history for debugging
README.md	Setup instructions and photography tips

1.2 Quick Start

```
cd artsy-phartsy-uploader
cp .env.example .env # fill in your credentials
npm install
npm start
```

Then drop a photo into one of the three input folders. Press Ctrl+C to stop gracefully.

1.3 Input Folders and Pricing

Folder	Item Type	Default Price
input/regular/	Light fabric bags	\$20.00
input/heavy/	Heavy fabric bags	\$25.00
input/aprons/	Aprons	\$15.00

Configuring Prices
All three prices are set in .env via PRICE_HEAVY, PRICE_LIGHT, and PRICE_APRON. No code changes needed to adjust pricing.

2. The Four-Step Pipeline

Every photo goes through these four steps in sequence. Each step has fallback behaviour — if it fails, the tool logs a warning and continues rather than crashing.

Step 1 — AI Item Detection (`detectBags` in `ai.js`)

The photo is sent to GPT-4o (high detail) which returns for each item:

- A creative "Miss ____" product name reflecting the fabric pattern and vibe
- A 1-2 sentence product description highlighting the upcycled fabric
- Normalized bounding box coordinates (x, y, width, height as 0.0–1.0 fractions)

Each name is then checked against WooCommerce for uniqueness via a whimsical variant chain:

- "Miss X" → "Miss X Too" → "Miss X's Sister" → "Miss X's Twin" → "Miss X's Cousin" → "Miss X's Doppelganger" → "Miss X's Long Lost Cousin"
- If all variants are taken, GPT-4o generates a completely new name

Step 2 — Smart Cropping (`cropBags` in `ai.js`)

EXIF Auto-Rotation

`sharp().rotate()` is called before cropping to apply phone EXIF orientation data. Without this, bags photographed in portrait mode appear sideways in the output.

Smart Per-Side Padding

Rather than a fixed padding on all sides, the algorithm checks each neighbouring bag bounding box and limits padding to 45% of the available gap — preventing crops from bleeding into adjacent bags in batch photos.

For each side: `padding = min(maxPadding=10%, gapToNeighbour * 0.45)`

Photography Tip

Leave 2-3 inches of backdrop between bags in batch photos. When bags touch or overlap, even smart padding cannot prevent bleed-through since the AI bounding boxes themselves may overlap.

Step 3a — Bag Puffing (`puffBag` in `ai.js`)

The cropped flat bag is sent to `gpt-image-2` to transform it into a 3D filled product shot. The prompt instructs the model to:

- Fill the bag with volume as if stuffed with a pillow
- Keep all fabric patterns, colors, and textures exactly as they appear
- Preserve the branding tag at the top center between handles
- Apply a randomly selected presentation style (see below)
- Use a clean neutral light gray studio background

Presentation Variety

To prevent all bags looking identical, puffBag randomly selects one of six handle/angle styles:

- Upright with handles standing tall
- Upright with handles flopped forward over the bag
- Slightly angled left with handles standing
- Slightly angled right with one handle flopped, one standing
- Upright with both handles drooping to each side
- Three-quarter angle with handles flopped forward

□ What Changed

NEW in v2: gpt-image-2 now correctly preserves the "ARTSY PHARTSY / OLD MADE NEW" branding tag text. gpt-image-1 would always generate garbled text regardless of prompt instructions. gpt-image-2's reasoning step means it actually processes the instruction before generating.

Step 3b — Apron Rendering (renderApron in ai.js)

For aprons, gpt-image-2 places the apron on a professional faceless dress form mannequin. The detectTag + replaceTag system verifies tag integrity after rendering and composites the original tag back if the AI distorted it.

Step 3c — Background Replacement (prepareSecondaryImage in ai.js)

The original flat crop is sent to gpt-image-2 to replace the backdrop with a clean neutral light gray studio background, matching the puffed hero image. Falls back to a simple color-boost if the API call fails.

□ What Changed

NEW in v2: Both product images now have consistent clean studio backgrounds regardless of what backdrop was used during photography. This means photographers can focus on good bag lighting rather than worrying about the background.

Step 4 — WooCommerce Upload (woocommerce.js)

- Primary image (puffed) uploaded to WordPress media library
- 1500ms delay after primary upload (WordPress needs time to process the image before it can be attached to a product)
- Secondary image (background-replaced flat crop) uploaded
- Brand story generated by GPT-4o with slight variation per bag
- Draft product created with name, short description (brand story), long description, price, SKU, category, and both images

□ What Changed

NEW in v2: If primary image upload fails after all retries, secondary upload and product creation are skipped entirely. This prevents orphaned draft products with no images or mismatched image pairs.

3. Key Files in Detail

3.1 ai.js — All AI Operations

Function	What it does
detectBags()	GPT-4o high-detail: finds items, names them "Miss ____", returns bounding boxes
cropBags()	sharp: EXIF rotate, smart-pad crop each item from batch photo
generateAlternativeName()	GPT-4o: generates fresh name when all whimsical variants are taken
generateBrandStory()	GPT-4o: creates varied brand story JSON with field validation
detectTag()	GPT-4o: locates the branding tag in an image, returns bbox
replaceTag()	sharp: composites original tag onto rendered image if AI distorted it (aprons)
puffBag()	gpt-image-2: transforms flat bag into 3D puffed product shot with random presentation style
renderApron()	gpt-image-2: places apron on dress form mannequin
prepareSecondaryImage()	gpt-image-2: replaces background with clean studio gray; falls back to color boost
generateSKU()	Generates unique SKU: AP-XXXXXX-YYYYZZ format

3.2 woocommerce.js — Store Integration

Function	What it does
checkNameExists()	Searches WooCommerce for exact name match across all statuses
ensureUniqueName()	Runs whimsical variant chain until a unique name is found
uploadImage()	Uploads to WordPress media library; auto-compresses PNG/large files; retries up to 3x on timeout
createDraftProduct()	Creates WooCommerce draft with full product data and HTML brand story
uploadBag()	Orchestrates image uploads then product creation; skips secondary and product if primary fails
getDraftsUrl()	Returns WP Admin URL for the drafts listing page (used in notification email)

3.3 Brand Story HTML Structure

Each product's short_description (shown in the right panel above Add to Cart) is built as:

```
<p>{AI-generated bag description}</p>
<ul>
  <li><strong>Occupation:</strong> Shopping Bag</li>
  <li><strong>Handmade by:</strong> {varied text}</li>
  <li><strong>Handmade from:</strong> {varied text}</li>
```

```
<li><strong>Quality:</strong> {varied text}</li>
<li><strong>Purpose:</strong> {varied text} <a href="...">Learn more</a></li>
<li><strong>Bag Dimensions:</strong> 14 inches wide, 15 inches high and 5 inches
deep</li>
</ul>
```

Brand Story Validation

generateBrandStory() validates that the AI returned all five expected field names (occupation, handmadeBy, handmadeFrom, quality, purpose). If any field is missing — which can happen when the AI decides to rename fields — it logs a warning and falls back to default text rather than showing "undefined" on the store.

4. Environment Variables (.env)

Copy .env.example to .env and fill in your values. This file must never be committed to git — it is listed in .gitignore.

Variable	Purpose
OPENAI_API_KEY	OpenAI API key — needs access to GPT-4o and gpt-image-2
WOO_URL	Your WooCommerce store URL (e.g. https://yourstore.com)
WOO_CONSUMER_KEY	WooCommerce REST API consumer key (ck_...)
WOO_CONSUMER_SECRET	WooCommerce REST API consumer secret (cs_...)
WP_USERNAME	WordPress username for media uploads
WP_APP_PASSWORD	WordPress Application Password (Settings > Users > Application Passwords)
SMTP_HOST	Email server hostname
SMTP_PORT	Email server port (587 for TLS)
SMTP_USER	Email address for sending notifications
SMTP_PASS	Email password or app password
NOTIFY_EMAILS	Comma-separated list of notification recipients
PRICE_HEAVY	Heavy bag price in dollars (default: 25)
PRICE_LIGHT	Light bag price in dollars (default: 20)
PRICE_APRON	Apron price in dollars (default: 15)
INPUT_FOLDER	Root input folder path (default: ./input)
OUTPUT_FOLDER	Processed file output path (default: ./output)
FILE_SETTLE_MS	Wait time after file detected before processing (default: 2000)

Two Authentication Systems

WooCommerce REST API uses WOO_CONSUMER_KEY + WOO_CONSUMER_SECRET. WordPress media upload uses a separate Application Password (WP_APP_PASSWORD) — these are different systems on the same WordPress install. The Application Password is created in WP Admin under Users > Profile > Application Passwords.

5. Key Design Decisions

5.1 gpt-image-2 vs gpt-image-1

The project originally used gpt-image-1. Despite extensive prompt engineering, gpt-image-1 consistently generated garbled tag text ("STEELRUBEL", "BAS MALN HBS", etc.) on every puffed image. Workarounds attempted included:

- Stronger prompt instructions — no effect
- Explicitly stating the exact tag text — no effect
- AI-based tag detection and blur overlay — partially worked but unreliable positioning
- Fixed-zone blur — consistent but covered too much or too little of the bag

Switching to gpt-image-2 (available April 2026) resolved the issue immediately. Its reasoning capability means it processes the tag preservation instruction contextually rather than just generating plausible-looking pixels.

Key Lesson

For tasks requiring precise instruction following — especially text preservation — gpt-image-2 is significantly more reliable than gpt-image-1. The reasoning step is the difference between "generates plausible output" and "actually follows the instruction".

5.2 Draft Products, Not Published

All products are created as WooCommerce drafts. This gives the store owner a review step before anything goes live — useful since AI occasionally makes mistakes with names, descriptions, or if a batch photo didn't separate cleanly. The owner can quickly scan drafts, make minor adjustments, and publish.

5.3 The Image Upload Race Condition

When an image is uploaded to WordPress, the API returns a media ID immediately but continues processing in the background. If a product is created too quickly, WordPress rejects the media ID as invalid. A 1500ms delay after each primary image upload prevents this reliably.

5.4 Smart Padding vs Fixed Padding

Fixed padding (e.g. always 10%) worked for single-bag photos but caused crops to bleed into neighbours in batch photos. The smart padding algorithm constrains each side to 45% of the gap to the nearest neighbour — single bags get generous padding, tightly-packed bags get proportionally less, automatically.

5.5 Brand Story Field Validation

GPT-4o occasionally returns JSON with different field names than specified (e.g. `handmade_by` instead of `handmadeBy`). The fix validates the five required field names after parsing and falls back to hardcoded defaults if any are missing — preventing "undefined" from appearing on product listings.

5.6 WooCommerce Category

New products are assigned to category ID 22 ("Newest Products") by default. This is hardcoded in `createDraftProduct()` in `woocommerce.js`. Update this constant if your store uses a different category ID.

6. API Costs

Each product processed uses approximately:

Operation	Model	Approx. Cost
Bag detection + naming	GPT-4o (high detail)	~\$0.01
Brand story generation	GPT-4o	~\$0.005
Bag puffing	gpt-image-2	~\$0.03-0.04
Background replacement	gpt-image-2	~\$0.03-0.04
Total per bag	—	~\$0.07-0.10

For a backlog of 100 bags, expect approximately \$7-10 in OpenAI API costs.

Apron Cost Note

Aprons use an additional detectTag() call (GPT-4o) before rendering plus a potential replaceTag() composite operation. This adds approximately \$0.01-0.02 per apron over the bag cost.

7. Known Limitations

7.1 Tightly-Packed Batch Photos

When bags are photographed very close together with minimal gap, GPT-4o bounding boxes may overlap, causing crops to bleed into neighbouring bags. The smart padding algorithm reduces this but cannot fully compensate when bags are touching. Recommendation: leave 2-3 inches of backdrop between bags in batch photos.

7.2 Background Replacement Accuracy

The background replacement works best when the bag has good contrast against the original backdrop. If the bag fabric is the same color as the background, gpt-image-2 may have difficulty finding the edge and could alter the bag itself. Good even lighting also helps.

7.3 Apron Pipeline Not Fully Tested

The apron pipeline (mannequin rendering + tag restoration) was implemented but not extensively tested. Run a test apron photo before processing a large backlog.

7.4 Network Errors

Image uploads retry up to 3 times on timeout (ETIMEDOUT) or server errors (5xx). Database connection errors from the hosting platform are not retryable — affected photos are moved to processed/failed/ and can be reprocessed manually.

7.5 AI Reliability

AI models are probabilistic, not deterministic. Occasional unexpected behaviours include: brand story returning different field names (handled by validation), puffing failing on a connection error (falls back to flat crop), bounding boxes being slightly inaccurate on complex batch photos. The tool has fallback behaviour for all known failure modes.

8. Troubleshooting

8.1 Checking the Log

```
tail -50 ~/artsy-phartsy/logs/uploader.log
grep "Miss Bag Name" ~/artsy-phartsy/logs/uploader.log
```

8.2 Reprocessing Failed Photos

Photos that failed are moved to input/[folder]/processed/failed/. Copy them back to retry:

```
cp ~/artsy-phartsy/input/heavy/processed/failed/photo.jpg ~/artsy-phartsy/input/heavy/
```

8.3 Common Errors

Error	Fix
unsupported mimetype (application/octet-stream)	The tmp PNG path regex failed. Verify croppedPath has a standard file extension.
ETIMEDOUT on image upload	Automatic retry fires up to 3 times. If all fail, photo goes to processed/failed/ — reprocess manually.
Error establishing a database connection	WordPress hosting issue. Wait a few minutes and reprocess.
#xxx is an invalid image ID	Race condition — increase the 1500ms delay in uploadBag() if this recurs.
Brand story fields showing "undefined"	AI returned unexpected field names. The validation fallback should catch this — check logs for "Brand story missing fields" warning.
Puffing failed — using cropped original	OpenAI API error. Product created with flat crop. Reprocess if puffed image needed.
name check failed — assuming unique	WooCommerce API timeout on name check. Check if name already exists manually.

8.4 Verifying the Correct Version is Running

```
grep "gpt-image" ~/artsy-phartsy/src/ai.js
```

Should only show gpt-image-2. If gpt-image-1 appears anywhere, redeploy the latest ai.js.

```
grep -n "blur" ~/artsy-phartsy/src/ai.js
```

Should return nothing. Any active blur code means an old version is deployed.

9. Dependencies

Package	Version	Purpose
openai	latest	GPT-4o + gpt-image-2 API client
sharp	latest	Image processing: rotate, crop, resize, compress, composite
chokidar	latest	Cross-platform file system watcher
axios	latest	HTTP client for WooCommerce and WordPress REST APIs
form-data	latest	Multipart form data for WordPress media upload
nodemailer	^8.0.8	Email notifications
winston	latest	Structured logging to console and file
dotenv	latest	Loads .env file into process.env

Node.js Requirement

Requires Node.js 20+ installed via nvm on WSL. The system must use the WSL Linux node binary — not a Windows node.exe on the PATH. Verify with: `which node`. If sharp fails to install, this is usually the cause.

10. Photography Guide

With background replacement now handled by AI, the focus for photographers shifts to lighting and framing rather than backdrop choice.

10.1 Lighting

- Bright, even lighting is the most important factor
- Natural daylight near a window on an overcast day is ideal — clouds act as a giant softbox
- Avoid direct harsh sunlight — creates strong shadows that confuse edge detection
- Avoid a single light source from one side — creates uneven lighting on the fabric
- Multiple soft light sources from different angles work well indoors

10.2 Framing

- Leave space above the handles and below the bag bottom in every shot
- For batch photos: leave 2-3 inches of backdrop between each item
- Maximum 3-4 items per batch photo for accurate AI separation
- The full bag must be visible — handles, body, and bottom edge

10.3 Background

- Background choice matters much less now — AI replaces it
- Still helpful to have good contrast between bag and backdrop
- Avoid photographing a dark bag against a dark background
- The original quilt or table backdrop is fine for most bags

11. Quick Reference

Daily Operations

- Start: `cd ~/artsy-phartsy && npm start`
- Drop photos into `input/regular/`, `input/heavy/`, or `input/aprons/`
- Watch the log: `tail -f logs/uploader.log`
- Email arrives when drafts are ready — review and publish in WP Admin
- Stop: `Ctrl+C` (waits for current job to finish)

AI Models Used

- GPT-4o — detection, naming, brand story, tag detection
- gpt-image-2 — puffing, apron rendering, background replacement
- Key advantage of gpt-image-2: reasoning capability means it actually follows tag preservation instructions

Verify Clean Deployment

- `grep "gpt-image" src/ai.js` — should only show `gpt-image-2`
- `grep -n "blur" src/ai.js` — should return nothing